

Lecture 7: Asynchronous programming

Presentations students, introduction to async

Willem Vanhulle

DevLab Rust 2025

Tuesday December 9, 2025

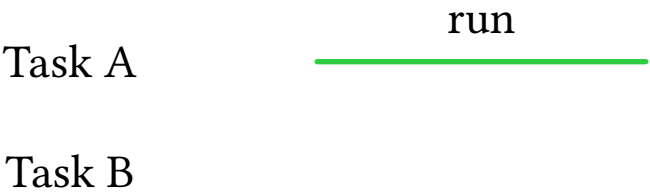
github.com/wvhulle/rust-course-ghent

1. Student presentations	1
1.1. Time-series transformers (Wietse)	2
1.2. Resource pool (Thomas)	2
2. Asynchronous programming	2
3. Asynchronous Rust	6
4. Rust's async implementation	10
5. Ecosystem	16
6. Common async datatypes and functions	19
7. Pitfalls	30
8. Exercise: async philosophers	35

1. Student presentations	1
2. Asynchronous programming	2
2.1. Introduction	3
2.2. Comparison with Python	4
2.3. Comparison with C++	5
3. Asynchronous Rust	6
4. Rust's async implementation	10
5. Ecosystem	16
6. Common async datatypes and functions	19
7. Pitfalls	30
8. Exercise: async philosophers	35

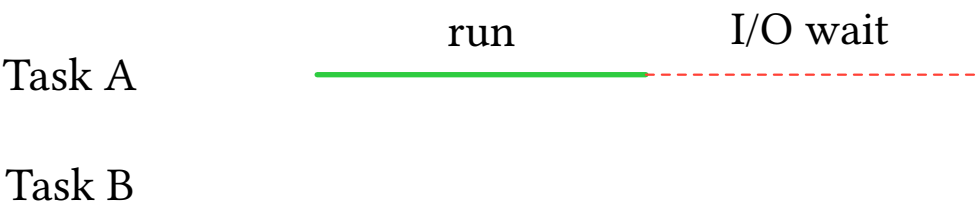
2.1. Introduction

Multiple tasks execute concurrently by running until they would block on I/O, then yielding to another ready task.



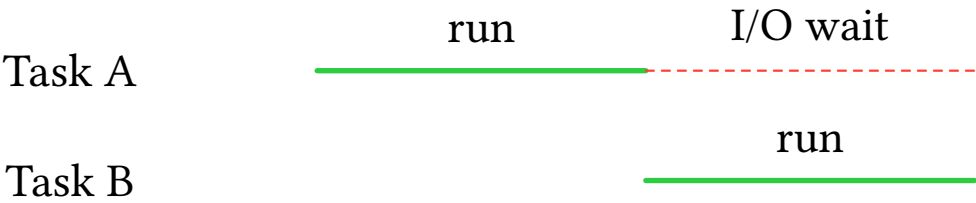
2.1. Introduction

Multiple tasks execute concurrently by running until they would block on I/O, then yielding to another ready task.

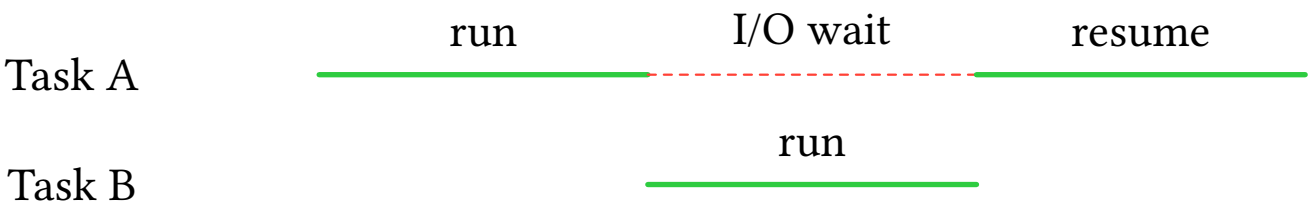


2.1. Introduction

Multiple tasks execute concurrently by running until they would block on I/O, then yielding to another ready task.

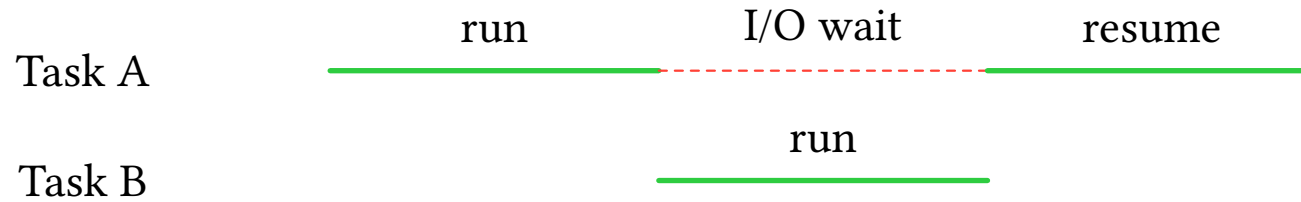


Multiple tasks execute concurrently by running until they would block on I/O, then yielding to another ready task.

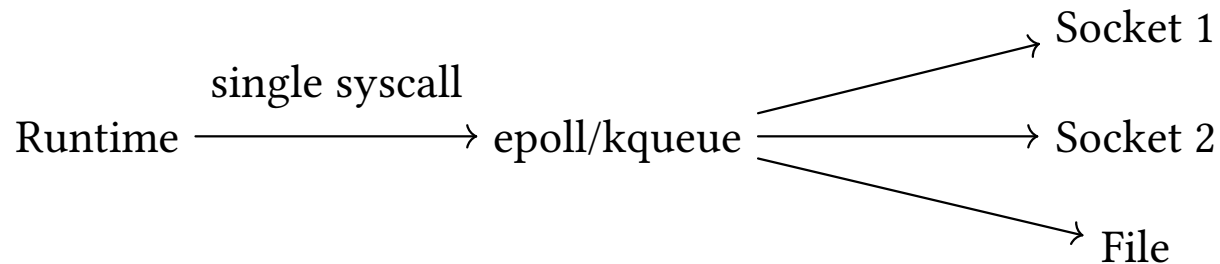


2.1. Introduction

Multiple tasks execute concurrently by running until they would block on I/O, then yielding to another ready task.

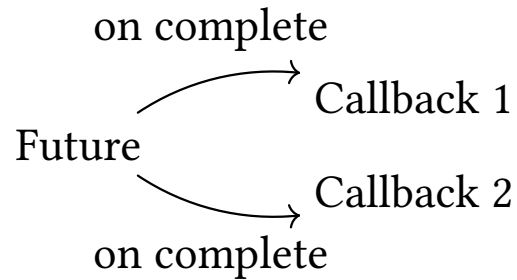


Per-task overhead is minimal because operating systems provide primitives (epoll, kqueue, IOCP) that monitor many I/O operations with a single system call.



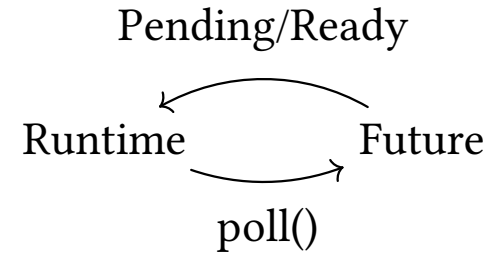
2.2. Comparison with Python

Python asyncio



- Callback-based model
- Register callbacks
- Built-in event loop
- Interpreter overhead

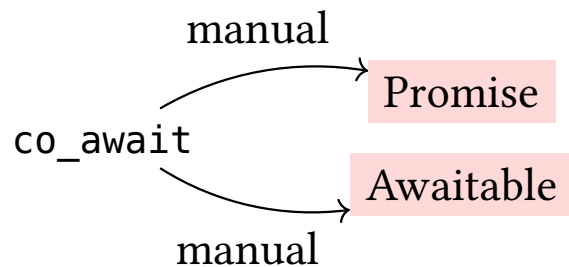
Rust async/await



- Poll-based model
- Runtime polls futures
- External runtimes (tokio, async-std)
- Zero-cost state machines

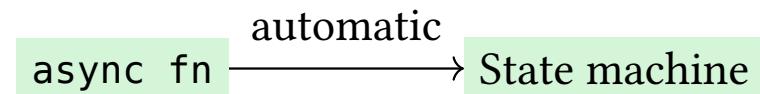
2.3. Comparison with C++

C++20 Coroutines



- Manual promise types
- Manual awaitable objects
- No standard runtime
- **Heap allocation** by default

Rust async/await



- Compiler generates state machines
- Built-in syntax
- Mature ecosystem (tokio)
- **Stack-allocated** by default

1.	Student presentations	1
2.	Asynchronous programming	2
3.	Asynchronous Rust	6
3.1.	Await syntax	7
3.2.	Futures	8
3.3.	JoinHandle	9
4.	Rust's async implementation	10
5.	Ecosystem	16
6.	Common async datatypes and functions	19
7.	Pitfalls	30
8.	Exercise: async philosophers	35

3.1. Await syntax

At a high level, async Rust code looks very much like “normal” sequential code:

```
1  use futures::executor::block_on;
2
3  async fn count_to(count: i32) {
4      for i in 0..count {
5          println!("Count is: {i}!");
6      }
7  }
8
9  async fn async_main(count: i32) {
10     count_to(count).await;
11 }
12
13 fn main() {
14     block_on(async_main(10));
15 }
```

([playground link](#))

3.2. Futures

Future is a trait, implemented by objects that represent an operation that may not be complete yet. A future can be polled, and poll returns a Poll.

```
1 use std::pin::Pin; use std::task::Context;
2
3 pub trait Future {
4     type Output;
5     fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
6 }
7
8 pub enum Poll<T> {
9     Ready(T),
10    Pending,
11 }
```

[\(playground link\)](#)

What is the role of Context?

3.2. Futures

Future is a trait, implemented by objects that represent an operation that may not be complete yet. A future can be polled, and poll returns a Poll.

```
1  use std::pin::Pin; use std::task::Context;
2
3  pub trait Future {
4      type Output;
5      fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
6  }
7
8  pub enum Poll<T> {
9      Ready(T),
10     Pending,
11 }
```

[\(playground link\)](#)

What is the role of Context?

Context allows a Future to schedule itself to be polled again when an event such as a timeout occurs.

3.3. JoinHandle

One common type implementing Future is a Tokio JoinHandle:

```
1 use tokio::task;
2
3 let join = task::spawn(async {
4     panic!("something bad happened!");
5 });
6
7 // The returned result indicates that the task failed.
8 assert!(join.await.is_err());
```

[*\(playground link\)*](#)

1.	Student presentations	1
2.	Asynchronous programming	2
3.	Asynchronous Rust	6
4.	Rust's async implementation	10
4.1.	State machine	11
4.2.	Stackless vs Stackful	12
4.3.	Transformation	13
4.4.	State machine structure	14
4.5.	Recursion	15
5.	Ecosystem	16
6.	Common async datatypes and functions	19
7.	Pitfalls	30
8.	Exercise: async philosophers	35

4.1. State machine

Rust transforms async functions into state machines that implement Future.

4.1. State machine

Rust transforms async functions into state machines that implement Future.

Key characteristics:

- Calling an async function constructs and returns a future
- Does not execute immediately (lazy)
- Tracks progress through suspension points

4.1. State machine

Rust transforms async functions into state machines that implement Future.

Key characteristics:

- Calling an async function constructs and returns a future
- Does not execute immediately (lazy)
- Tracks progress through suspension points

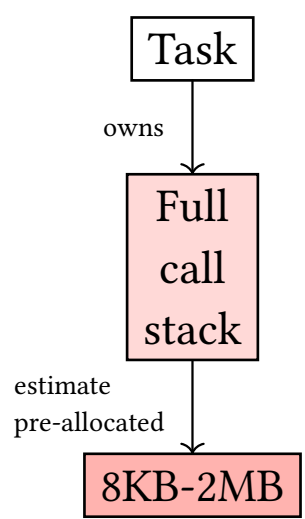
The state machines are allocated on the stack by default. Once allocated, the async runtime never moves the state machine to a different memory location.

Definition

Rust uses **stackless coroutines**: async functions don't have their own call stack. Instead, they compile to state machines that store only the data needed at suspension points, enabling zero-cost abstractions.

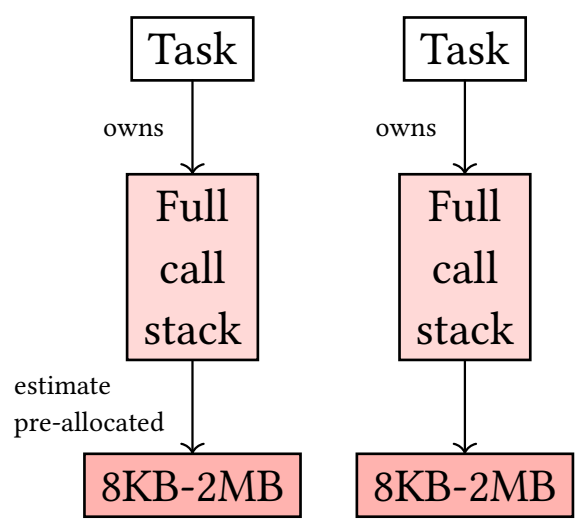
4.2. Stackless vs Stackful

Stackful (Go, goroutines)



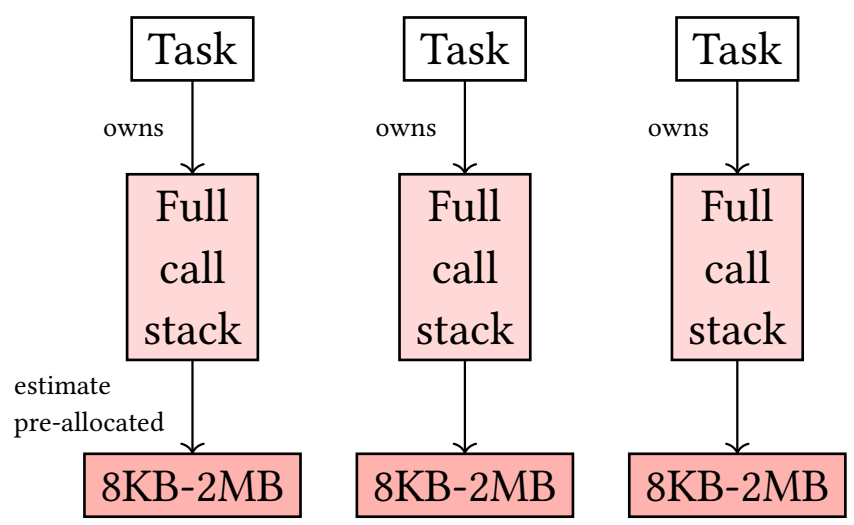
4.2. Stackless vs Stackful

Stackful (Go, goroutines)



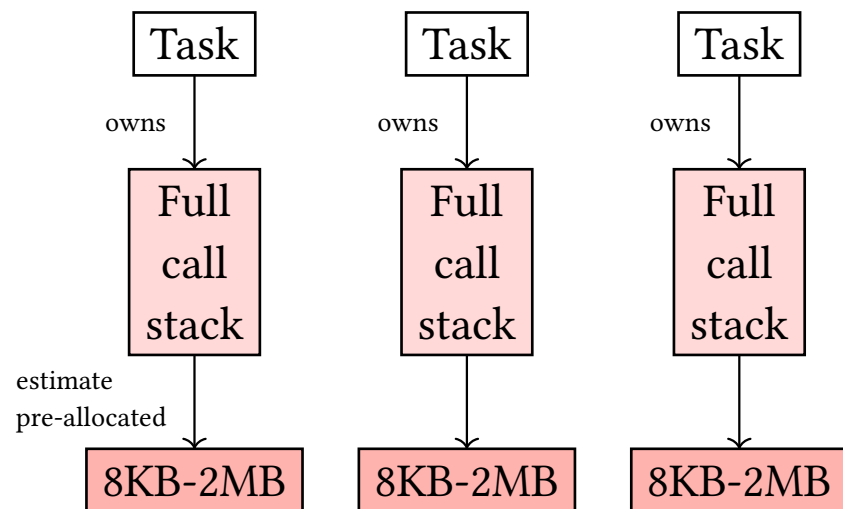
4.2. Stackless vs Stackful

Stackful (Go, goroutines)



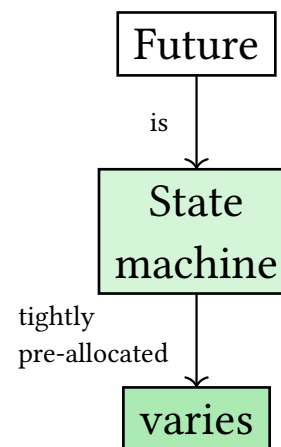
4.2. Stackless vs Stackful

Stackful (Go, goroutines)



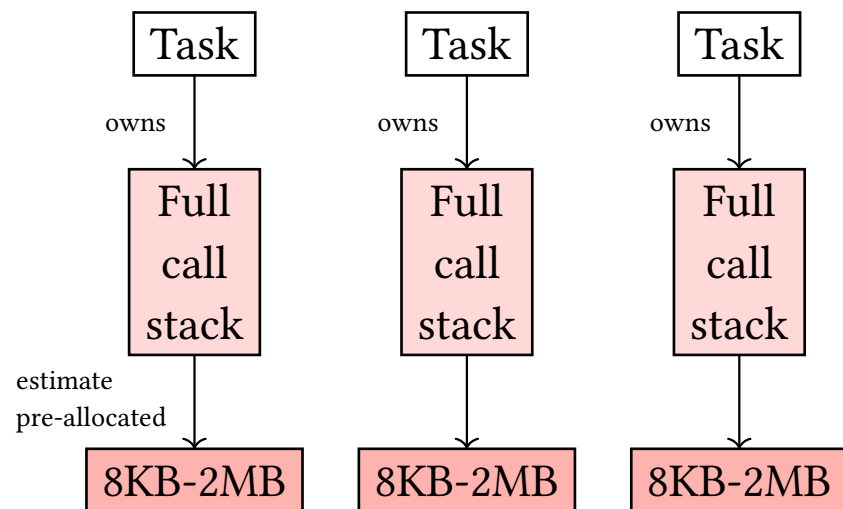
10K tasks = 80MB-20GB

Stackless (Rust async)



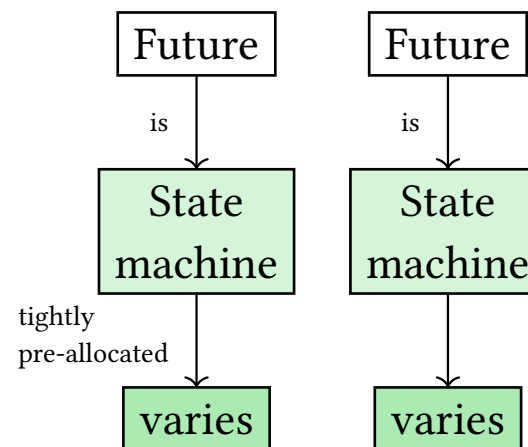
4.2. Stackless vs Stackful

Stackful (Go, goroutines)



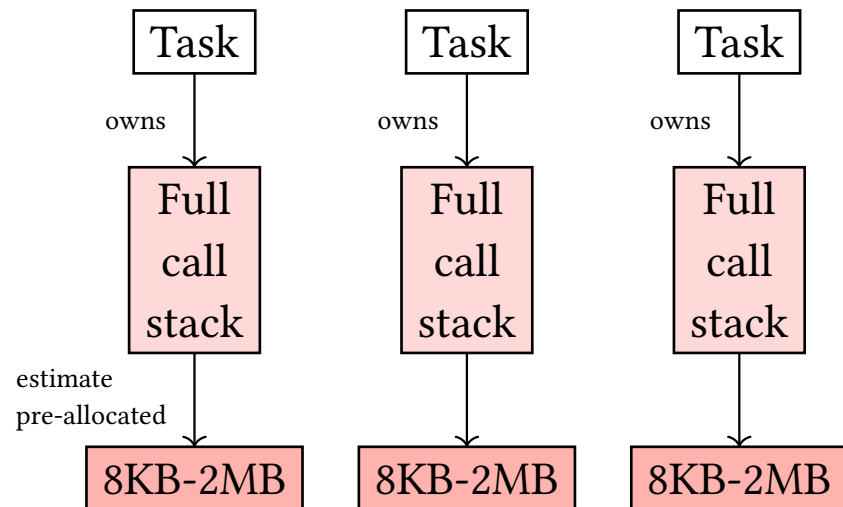
10K tasks = 80MB-20GB

Stackless (Rust async)



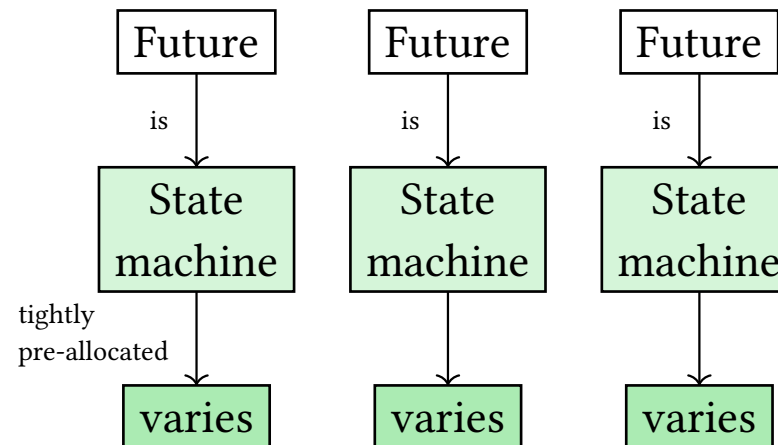
4.2. Stackless vs Stackful

Stackful (Go, goroutines)



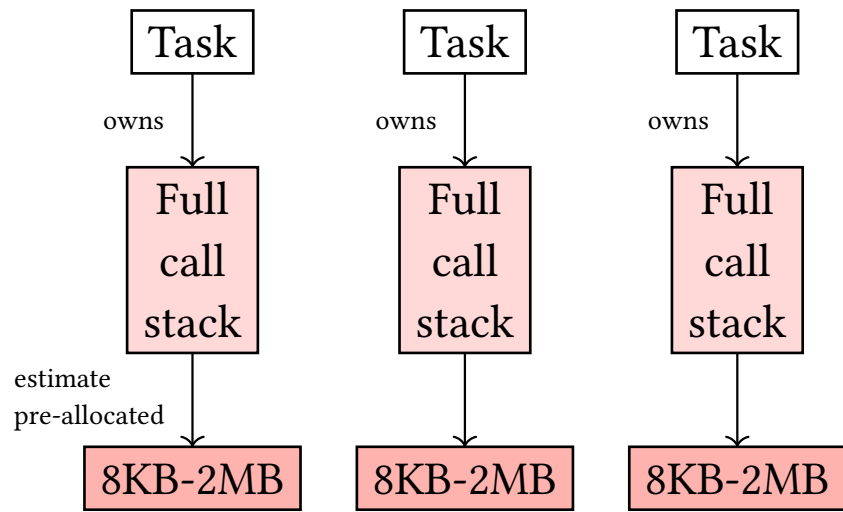
10K tasks = 80MB-20GB

Stackless (Rust async)



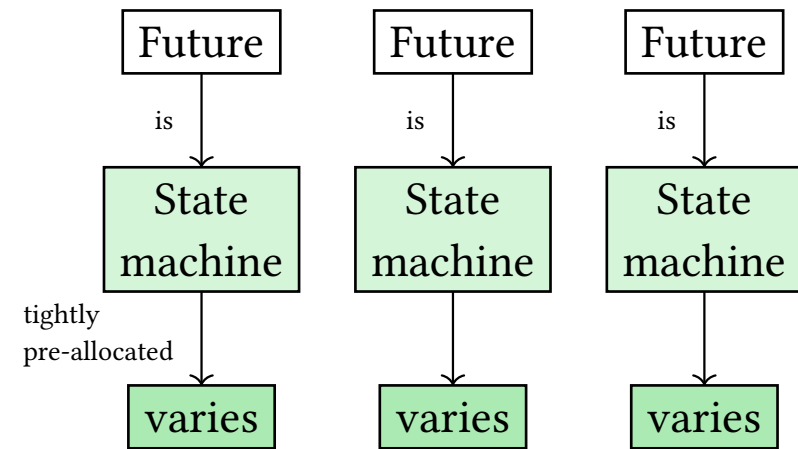
4.2. Stackless vs Stackful

Stackful (Go, goroutines)



10K tasks = 80MB-20GB

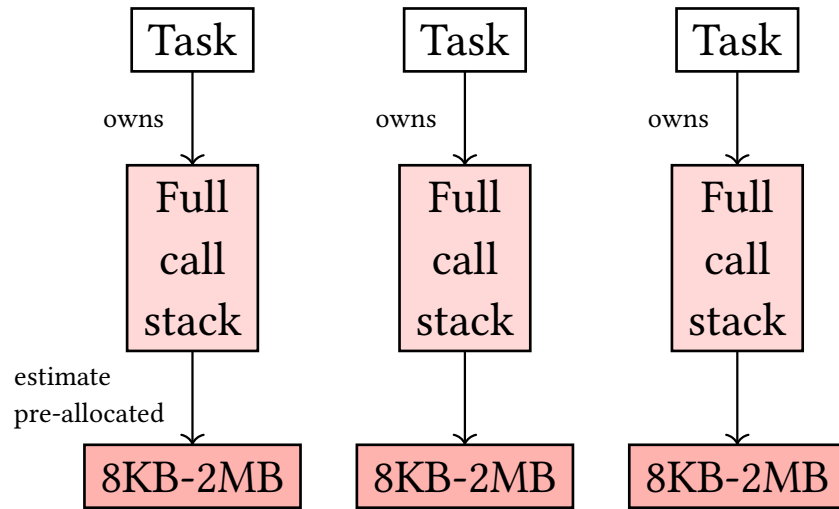
Stackless (Rust async)



10K tasks = 1MB

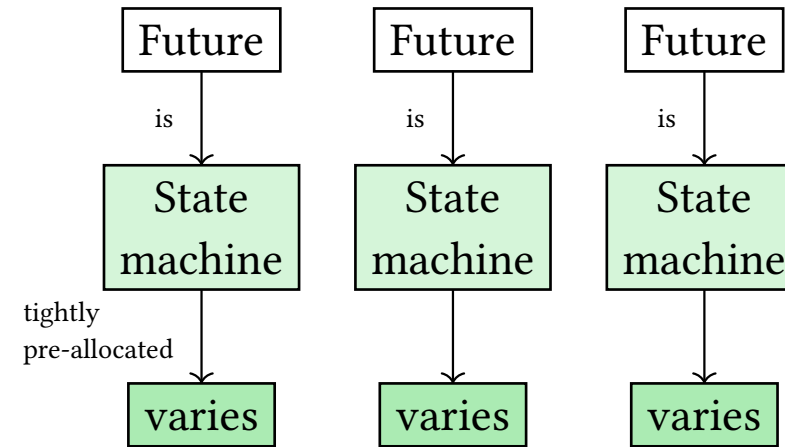
4.2. Stackless vs Stackful

Stackful (Go, goroutines)



10K tasks = 80MB-20GB

Stackless (Rust async)



10K tasks = 1MB

Key difference: Stackful tasks allocate a fixed-size stack (pre-allocated for deep call chains), while stackless state machines allocate only the exact space needed for live variables at each suspension point.

4.3. Transformation

Consider this async function:

```
1 async fn two_d10(modifier: u32) -> u32 {  
2     let first_roll = roll_d10().await;  
3     let second_roll = roll_d10().await;  
4     first_roll + second_roll + modifier  
5 }
```

[\(playground link\)](#)

4.3. Transformation

Consider this async function:

```
1 async fn two_d10(modifier: u32) -> u32 {  
2     let first_roll = roll_d10().await;  
3     let second_roll = roll_d10().await;  
4     first_roll + second_roll + modifier  
5 }
```

[\(playground link\)](#)

The compiler generates an enum that:

- Tracks each suspension point (each `.await`)
- Stores all live variables needed at that state
- Implements the Future trait

4.4. State machine structure

The generated enum looks conceptually like this:

```
1 enum TwoDiceFuture {  
2     Start { modifier: u32 },  
3     FirstRoll { modifier: u32, first_roll: impl Future<Output = u32> },  
4     SecondRoll { modifier: u32, first_result: u32, second_roll: impl Future<Output =  
5         u32> },  
6     Done,  
}
```

[\(playground link\)](#)

4.4. State machine structure

The generated enum looks conceptually like this:

```
1 enum TwoDiceFuture {  
2     Start { modifier: u32 },  
3     FirstRoll { modifier: u32, first_roll: impl Future<Output = u32> },  
4     SecondRoll { modifier: u32, first_result: u32, second_roll: impl Future<Output =  
5         u32> },  
6     Done,  
7 }
```

[\(playground link\)](#)

Each state stores:

- The variables still needed
- The future being awaited (if any)

Deeply nested async functions create large compiler-generated Future types.

Each function's Future contains its callees' Futures.

4.5. Recursion

Deeply nested async functions create large compiler-generated Future types.

Each function's Future contains its callees' Futures.

Recursive async functions require boxing to avoid infinite type sizes:

```
1  async fn count_to(n: u32) {  
2      if n > 0 {  
3          Box::pin(count_to(n - 1)).await;  
4          println!("{n}");  
5      }  
6  }  
7  
8  Box::pin(count_to(n - 1)).await;
```

([playground link](#))

4.5. Recursion

Deeply nested async functions create large compiler-generated Future types.

Each function's Future contains its callees' Futures.

Recursive async functions require boxing to avoid infinite type sizes:

```
1 async fn count_to(n: u32) {  
2     if n > 0 {  
3         Box::pin(count_to(n - 1)).await;  
4         println!("{n}");  
5     }  
6 }  
7  
8 Box::pin(count_to(n - 1)).await;
```

([playground link](#))

This adds heap allocation overhead but makes recursion possible.

1.	Student presentations	1
2.	Asynchronous programming	2
3.	Asynchronous Rust	6
4.	Rust's async implementation	10
5.	Ecosystem	16
5.1.	Runtimes	17
5.2.	Tokio	18
6.	Common async datatypes and functions	19
7.	Pitfalls	30
8.	Exercise: async philosophers	35

5.1. Runtimes

A runtime provides support for:

- performing operations asynchronously (a **reactor**)
- and is responsible for executing futures (an **executor**).

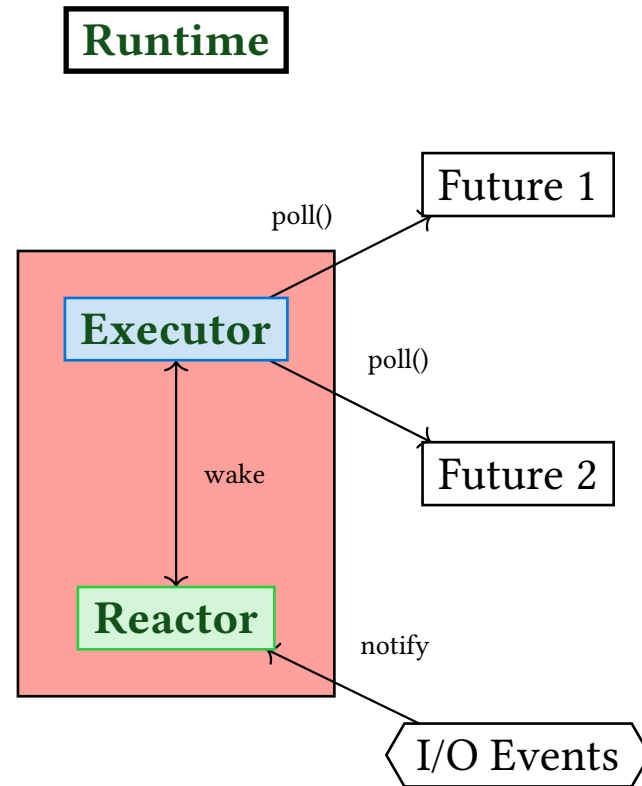
5.1. Runtimes

A runtime provides support for:

- performing operations asynchronously (a **reactor**)
- and is responsible for executing futures (an **executor**).

Rust does not have a “built-in” runtime, but several options are available:

- futures offers a bare bones executor ThreadPool
- tokio offers a Runtime with reactor included (supplies “time” etc.)



```
1 use tokio::time;
2 async fn count_to(count: i32) {
3     for i in 0..count {
4         println!("Count in task: {i}!");
5         time::sleep(time::Duration::from_millis(5)).await;
6     }
7 }
8 #[tokio::main]
9 async fn main() {
10     tokio::spawn(count_to(10));
11     for i in 0..5 {
12         println!("Main task: {i}");
13         time::sleep(time::Duration::from_millis(5)).await;
14     }
15 }
```

[\(playground link\)](#)

Async tasks are also aborted if not awaited in main.

1.	Student presentations	1
2.	Asynchronous programming	2
3.	Asynchronous Rust	6
4.	Rust's async implementation	10
5.	Ecosystem	16
6.	Common async datatypes and functions	19
6.1.	Tasks	20
6.2.	Exercise	23
6.3.	Channels	24
6.4.	Joining	25
6.5.	Select	27
6.6.	Streams	29
7.	Pitfalls	30
8.	Exercise: async philosophers	35

6.1. Tasks

6. Common async datatypes and functions

A task is a top-level future spawned into the runtime. In multi-threaded runtimes, tasks must be `Send + 'static` and may be moved between worker threads by the work-stealing scheduler.

```
1  #[tokio::main]
2  async fn main() -> io::Result<()> {
3      let listener = TcpListener::bind("127.0.0.1:0").await?;
4
5      loop {
6          let (mut socket, addr) = listener.accept().await?;
7
8          tokio::spawn(async move {
9              // Handle connection
10          });
11      }
12 }
```

[*\(playground link\)*](#)

6.1. Tasks

6. Common async datatypes and functions

A task is a top-level future spawned into the runtime. In multi-threaded runtimes, tasks must be `Send + 'static` and may be moved between worker threads by the work-stealing scheduler.

```
1  #[tokio::main]
2  async fn main() -> io::Result<()> {
3      let listener = TcpListener::bind("127.0.0.1:0").await?;
4
5      loop {
6          let (mut socket, addr) = listener.accept().await?;
7
8          tokio::spawn(async move {
9              // Handle connection
10          });
11      }
12 }
```

[\(playground link\)](#)

Warning

Although we use tokio in this section, the same functionality is available in most async runtimes.

Inside `tokio::spawn`:

```
1 socket.write_all(b"Who are you?\n").await?;  
2  
3 let mut buf = vec![0; 1024];  
4 let name_size = socket.read(&mut buf).await?;  
5 let name = std::str::from_utf8(&buf[..name_size])?.trim();  
6  
7 socket.write_all(format!("Thanks, {name}!\n").as_bytes()).await?;    (playground link)
```

Tokio Runtime

Executor

Tokio Runtime

Tokio Runtime

Executor

Global
Queue

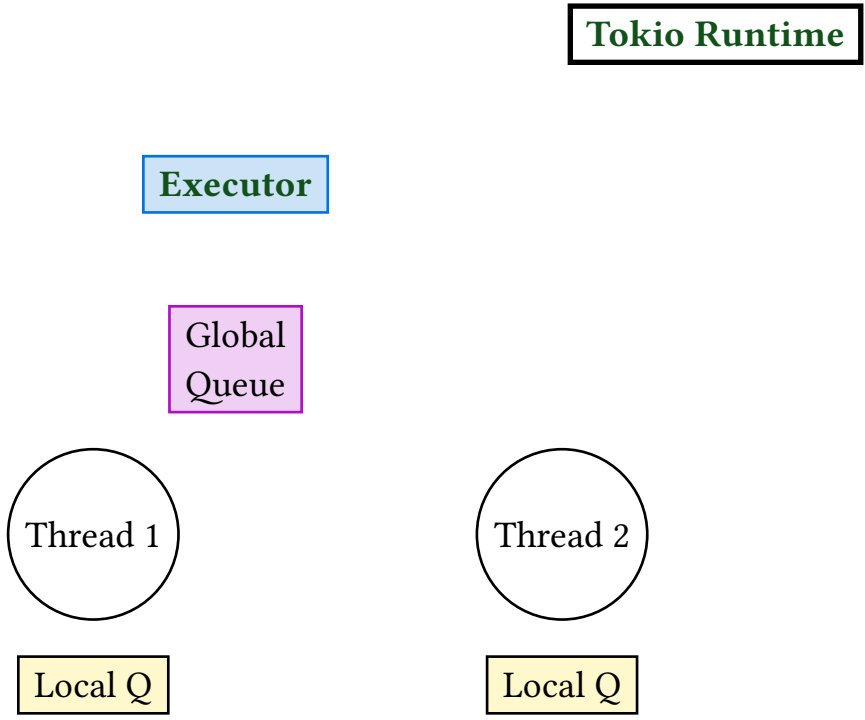
Tokio Runtime

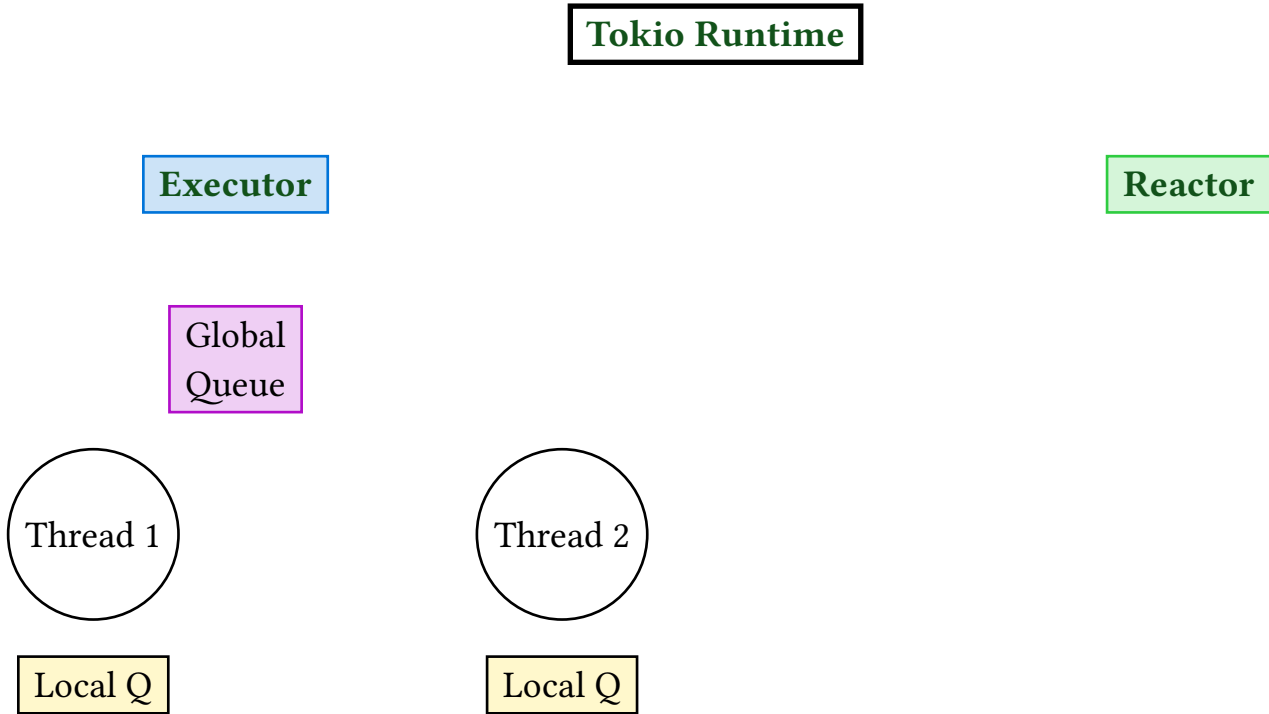
Executor

Global Queue

Thread 1

Local Q





Tokio Runtime

Executor

Reactor

Global Queue

Thread 1

T1

T4

Local Q

T2

Thread 2

Local Q

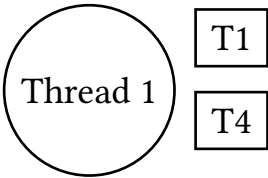
T3

Tokio Runtime

Executor

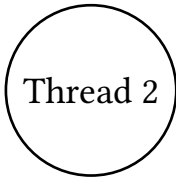
Reactor

Global Queue



Local Q

T2

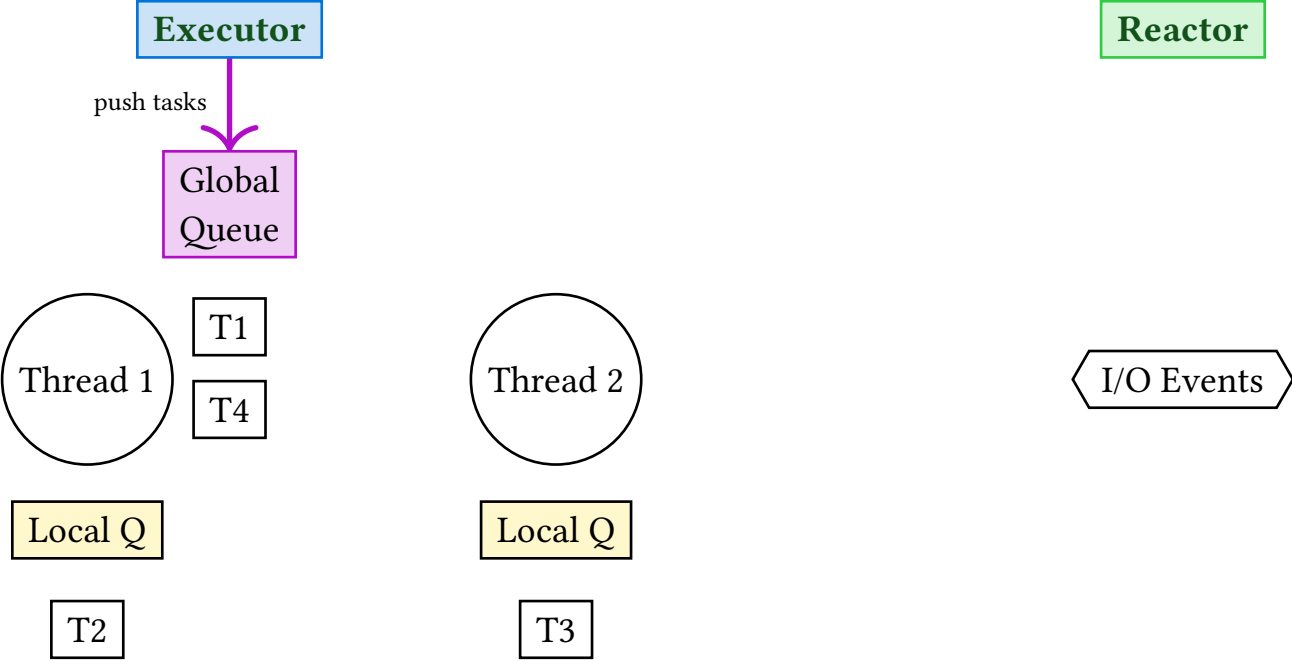


Local Q

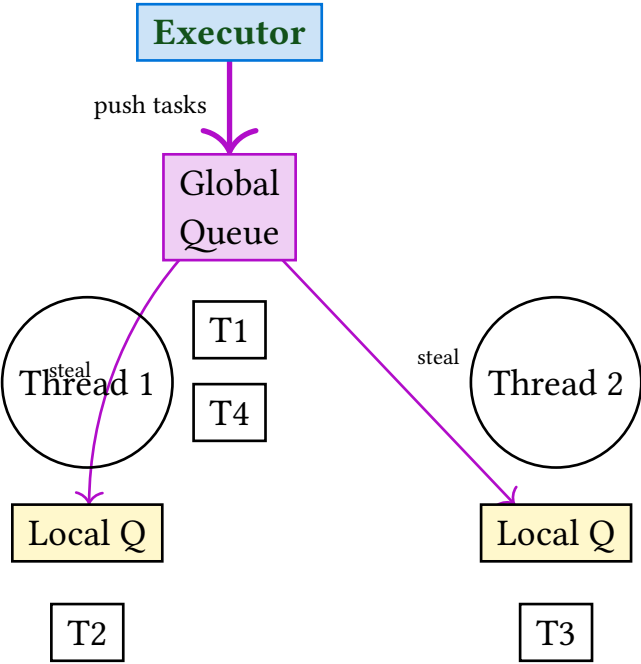
T3

I/O Events

Tokio Runtime



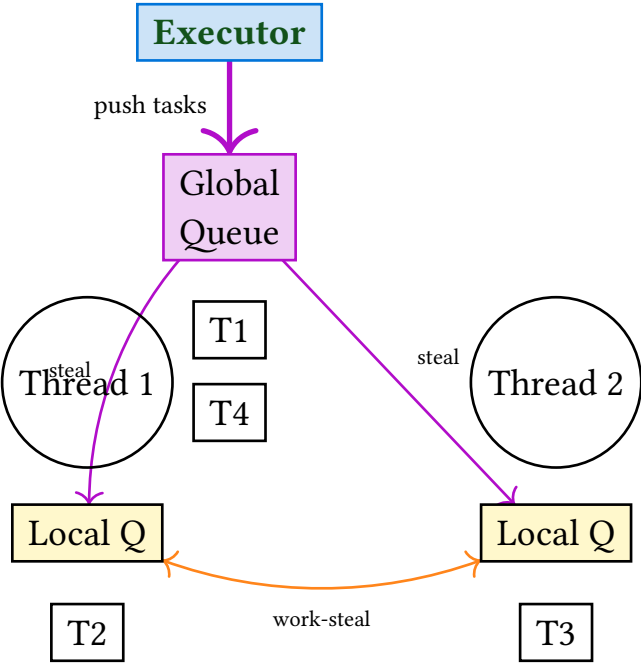
Tokio Runtime



Reactor

I/O Events

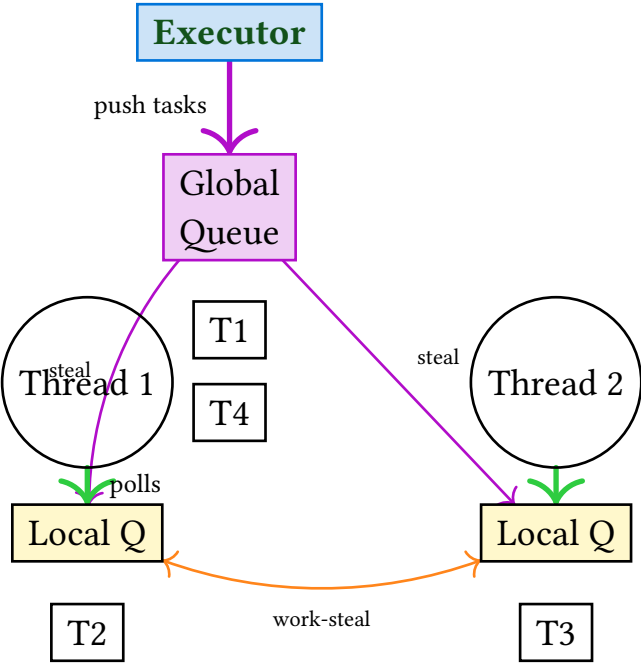
Tokio Runtime



Reactor

I/O Events

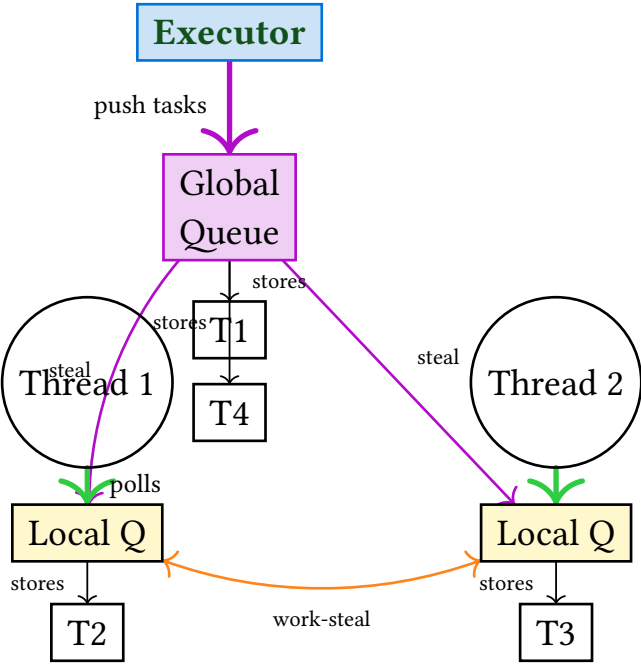
Tokio Runtime



Reactor

I/O Events

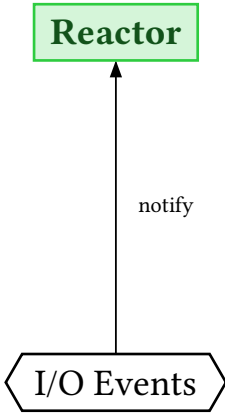
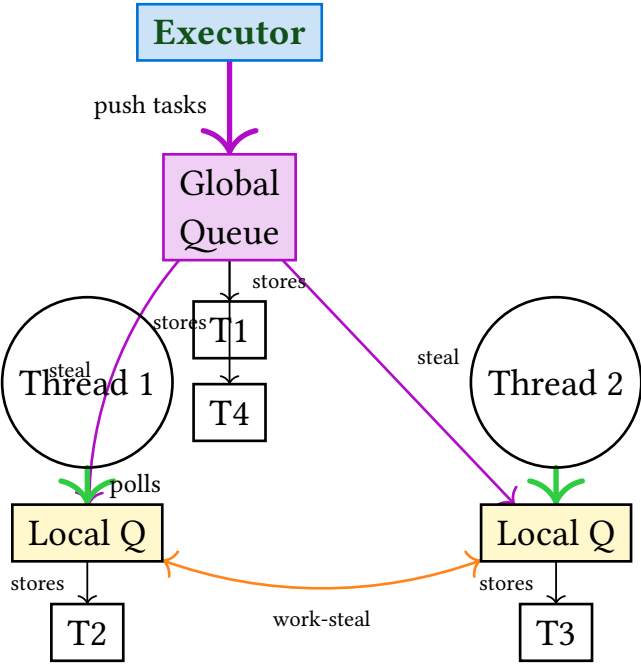
Tokio Runtime

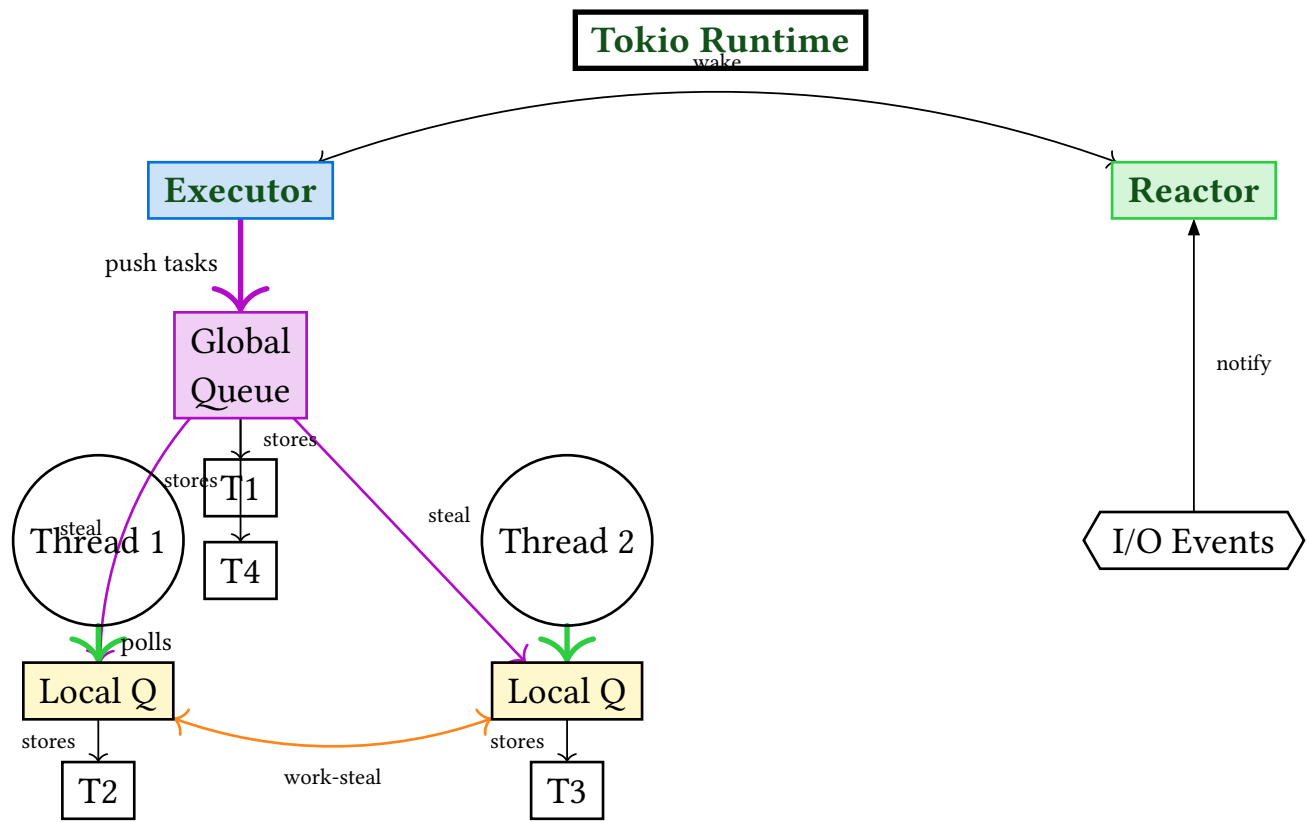


Reactor

I/O Events

Tokio Runtime





Info

An async block like `async move {}` is like a scope block that may contain awaits.

Refactor `session7/examples/tasks.rs`:

- Create async helper function from async block
- Improve error handling

```
1 async fn ping_handler(mut input: mpsc::Receiver<()>) {  
2     let mut count: usize = 0;  
3     while let Some(_) = input.recv().await {  
4         count += 1;  
5         println!("Received {count} pings so far.");  
6     }  
7 }
```

[\(playground link\)](#)

```
1 async fn ping_handler(mut input: mpsc::Receiver<()>) {
2     let mut count: usize = 0;
3     while let Some(_) = input.recv().await {
4         count += 1;
5         println!("Received {count} pings so far.");
6     }
7 }
```

[\(playground link\)](#)

```
1 async fn main() {
2     let (sender, receiver) = mpsc::channel(32);
3     let handler_task = tokio::spawn(ping_handler(receiver));
4     for i in 0..10 {
5         sender.send(()).await.expect("Failed to send");
6     }
7     drop(sender);
8     handler_task.await.expect("Handler failed");
9 }
```

[\(playground link\)](#)

We might need to do the same async function multiple times:

```
1 async fn size_of_page(url: &str) -> Result<usize> {  
2     let resp = request::get(url).await?;  
3     Ok(resp.text().await?.len())  
4 }
```

[\(playground link\)](#)

How can we await a list of futures concurrently?

6.4. Joining

We might need to do the same async function multiple times:

```
1 async fn size_of_page(url: &str) -> Result<usize> {  
2     let resp = reqwest::get(url).await?;  
3     Ok(resp.text().await?.len())  
4 }
```

[\(playground link\)](#)

How can we await a list of futures concurrently?

We use the `join_all` function.

6.4. Joining

We might need to do the same async function multiple times:

```
1 async fn size_of_page(url: &str) -> Result<usize> {  
2     let resp = request::get(url).await?;  
3     Ok(resp.text().await?.len())  
4 }
```

[\(playground link\)](#)

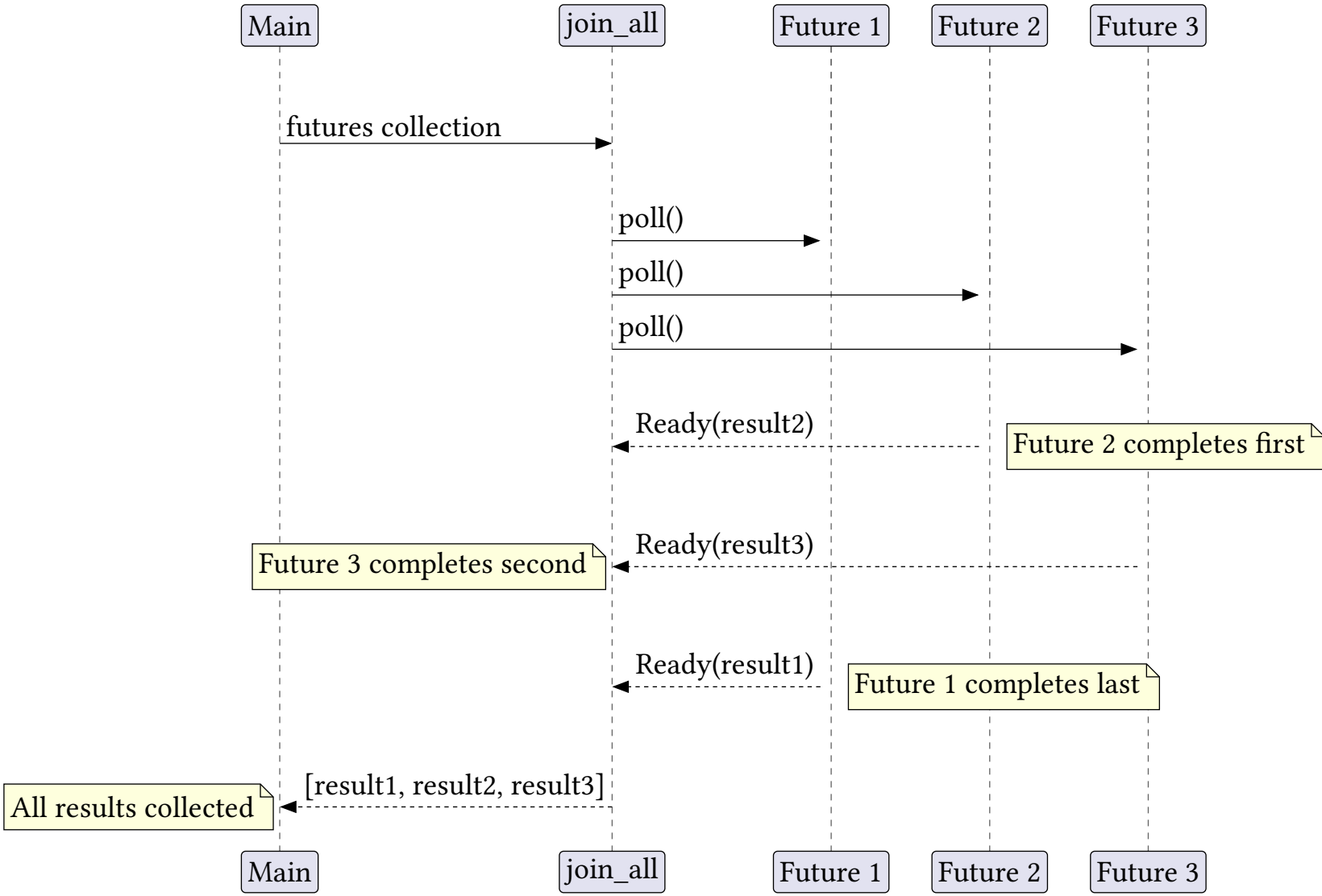
How can we await a list of futures concurrently?

We use the `join_all` function.

```
1 async fn main() {  
2     let urls = ["https://google.com", "https://httpbin.org/ip", /*...*/];  
3     let futures = urls.into_iter().map(size_of_page);  
4     let results = future::join_all(futures).await;  
5     println!("{results:?}");  
6 }
```

[\(playground link\)](#)

pagebreak

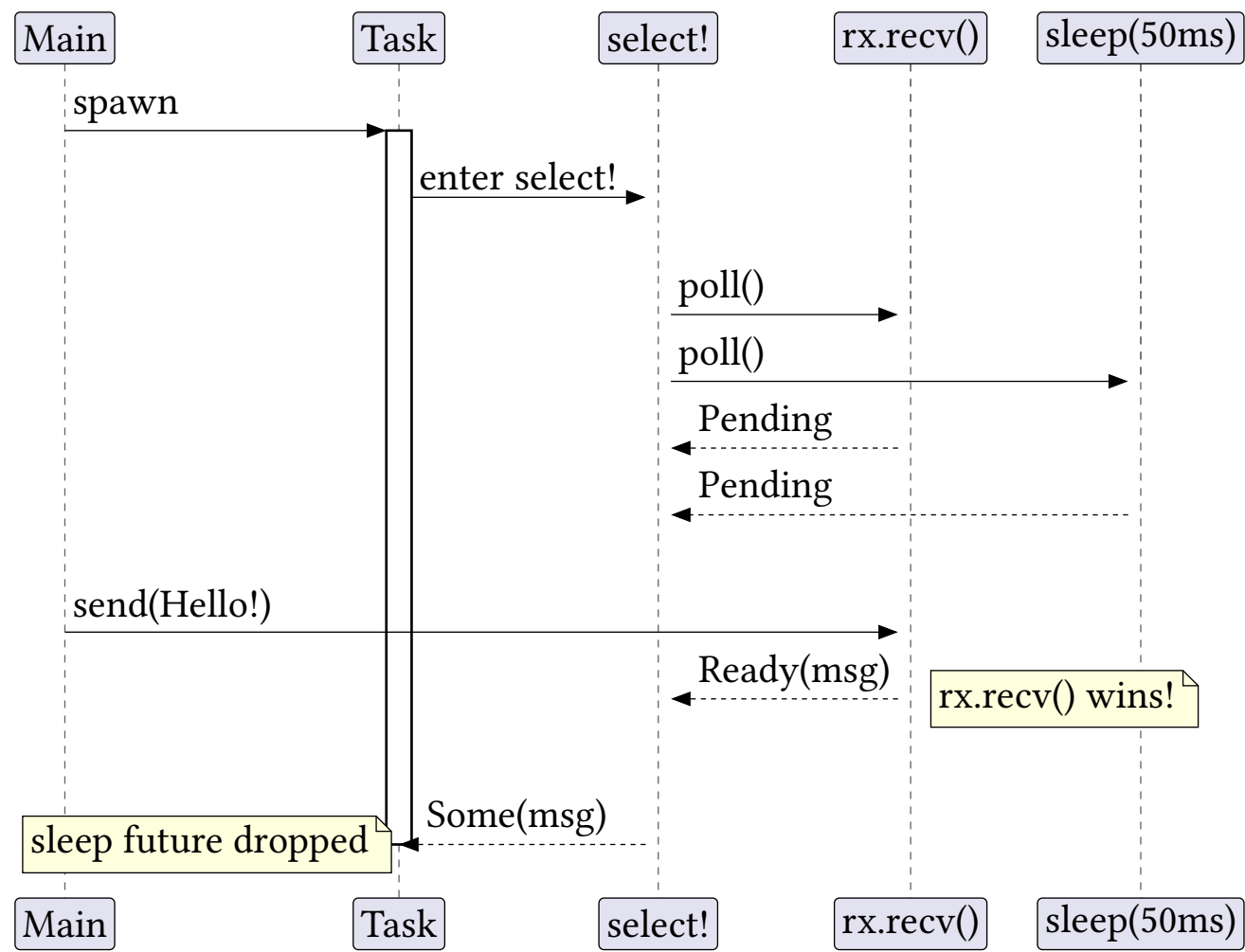


6.5. Select

Waits until any of a set of futures is ready. Similar to `Promise.race` or Python's `asyncio.wait()`.

```
1  #[tokio::main]
2  async fn main() {
3      let (tx, mut rx) = mpsc::channel(32);
4
5      tokio::spawn(async move {
6          tokio::select! {
7              Some(msg) = rx.recv() => println!("got: {msg}"),
8              _ = sleep(Duration::from_millis(50)) => println!("timeout"),
9          };
10     });
11
12     sleep(Duration::from_millis(10)).await;
13     tx.send(String::from("Hello!")).await?;
14     // ...
15 }
```

([playground link](#))



Streams are runtime-agnostic asynchronous iterators:

```
1 use futures::stream::{self, StreamExt};
2
3 let stream = stream::iter(vec!['a', 'b', 'c']);
4
5 let mut stream = stream.enumerate();
6
7 assert_eq!(stream.next().await, Some((0, 'a')));
8 assert_eq!(stream.next().await, Some((1, 'b')));
9 assert_eq!(stream.next().await, Some((2, 'c')));
10 assert_eq!(stream.next().await, None);
```

([playground link](#))

See my project <https://github.com/wvhulle/clone-stream>

1.	Student presentations	1
2.	Asynchronous programming	2
3.	Asynchronous Rust	6
4.	Rust's async implementation	10
5.	Ecosystem	16
6.	Common async datatypes and functions	19
7.	Pitfalls	30
7.1.	Blocking executor	31
7.2.	Async for limited parallelism	32
7.3.	Avoid spawn-ception	33
7.4.	Deadlocks: Coffman Conditions	34
8.	Exercise: async philosophers	35

7.1. Blocking executor

CPU blocking tasks will block the executor and prevent other tasks from being executed.

```
1  async fn sleep_ms(start: &Instant, id: u64, duration_ms: u64) {
2      std::thread::sleep(std::time::Duration::from_millis(duration_ms));
3      println!("future {id} finished after {}ms", start.elapsed().as_millis());
4  }
5
6  #[tokio::main(flavor = "current_thread")]
7  async fn main() {
8      let start = Instant::now();
9      let futures = (1..=10).map(|t| sleep_ms(&start, t, t * 10));
10     join_all(futures).await;
11 }
```

([playground link](#))

7.1. Blocking executor

CPU blocking tasks will block the executor and prevent other tasks from being executed.

```
1  async fn sleep_ms(start: &Instant, id: u64, duration_ms: u64) {
2      std::thread::sleep(std::time::Duration::from_millis(duration_ms));
3      println!("future {id} finished after {}ms", start.elapsed().as_millis());
4  }
5
6  #[tokio::main(flavor = "current_thread")]
7  async fn main() {
8      let start = Instant::now();
9      let futures = (1..=10).map(|t| sleep_ms(&start, t, t * 10));
10     join_all(futures).await;
11 }
```

([playground link](#))

What is a workaround?

7.1. Blocking executor

CPU blocking tasks will block the executor and prevent other tasks from being executed.

```
1  async fn sleep_ms(start: &Instant, id: u64, duration_ms: u64) {
2      std::thread::sleep(std::time::Duration::from_millis(duration_ms));
3      println!("future {id} finished after {}ms", start.elapsed().as_millis());
4  }
5
6  #[tokio::main(flavor = "current_thread")]
7  async fn main() {
8      let start = Instant::now();
9      let futures = (1..=10).map(|t| sleep_ms(&start, t, t * 10));
10     join_all(futures).await;
11 }
```

([playground link](#))

What is a workaround?

Use async methods: `tokio::time::sleep` instead of `std::thread::sleep`

7.2. Async for limited parallelism

Warning

Async tasks run on a limited thread pool and don't provide true parallelism for CPU-bound work.

Async is designed for I/O concurrency, not CPU parallelism:

- Spawning 10,000 async tasks doesn't create 10,000 workers
- Tasks execute sequentially on the available thread pool
- CPU-bound work sees no benefit from async spawning

7.2. Async for limited parallelism

Warning

Async tasks run on a limited thread pool and don't provide true parallelism for CPU-bound work.

Async is designed for I/O concurrency, not CPU parallelism:

- Spawning 10,000 async tasks doesn't create 10,000 workers
- Tasks execute sequentially on the available thread pool
- CPU-bound work sees no benefit from async spawning

Recommendation: If there's no real I/O waiting:

- Use a small number of OS threads instead (`std::thread` or `rayon`)
- Match thread count to CPU cores for optimal performance
- Reserve async for coordinating I/O operations

7.3. Avoid spawn-ception

Warning

Spawning tasks from within spawned tasks creates hard-to-debug complexity.

Common anti-pattern:

```
1 tokio::spawn(async {  
2     // Some work  
3     tokio::spawn(async { // Nested spawn!  
4         // More work  
5         tokio::spawn(async { /* ... */ })  
6     })  
7 })
```

[\(playground link\)](#)

7.3. Avoid spawn-ception

Warning

Spawning tasks from within spawned tasks creates hard-to-debug complexity.

Common anti-pattern:

```
1 tokio::spawn(async {  
2     // Some work  
3     tokio::spawn(async { // Nested spawn!  
4         // More work  
5         tokio::spawn(async { /* ... */ })  
6     })  
7 })
```

[\(playground link\)](#)

Recommendation: Spawn once at application boundaries:

- Spawn tasks at the top level (e.g., per incoming request)
- Use regular async functions and `.await` for sequential steps
- Only spawn when you need true concurrent execution

7.4. Deadlocks: Coffman Conditions

A deadlock occurs when four conditions are met simultaneously (Coffman conditions):

Condition	Description
Mutual exclusion	Resources cannot be shared (only one thread at a time)
Hold and wait	Threads hold resources while waiting for others
No preemption	Resources cannot be forcibly taken from threads
Circular wait	Threads form a cycle waiting for each other's resources

7.4. Deadlocks: Coffman Conditions

A deadlock occurs when four conditions are met simultaneously (Coffman conditions):

Condition	Description
Mutual exclusion	Resources cannot be shared (only one thread at a time)
Hold and wait	Threads hold resources while waiting for others
No preemption	Resources cannot be forcibly taken from threads
Circular wait	Threads form a cycle waiting for each other's resources

To prevent deadlock, break at least one condition:

- **Lock ordering:** Always acquire locks in the same order (breaks circular wait)
- **Timeout:** Release locks if waiting too long (breaks hold and wait)
- **Try-lock:** Use `try_lock()` instead of `lock()` (breaks hold and wait)

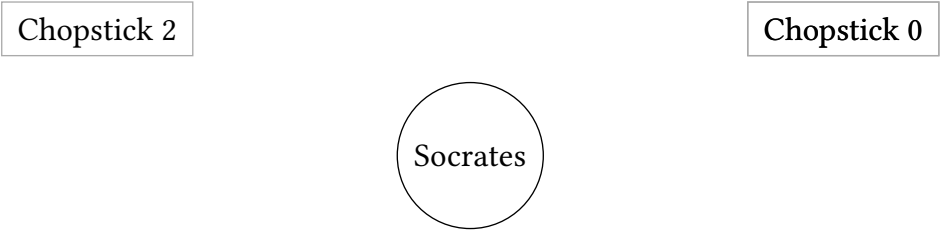
1.	Student presentations	1
2.	Asynchronous programming	2
3.	Asynchronous Rust	6
4.	Rust's async implementation	10
5.	Ecosystem	16
6.	Common async datatypes and functions	19
7.	Pitfalls	30
8.	Exercise: async philosophers	35
8.1.	Context	36
8.2.	Assignment	37
8.3.	Additional bonus	38

8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

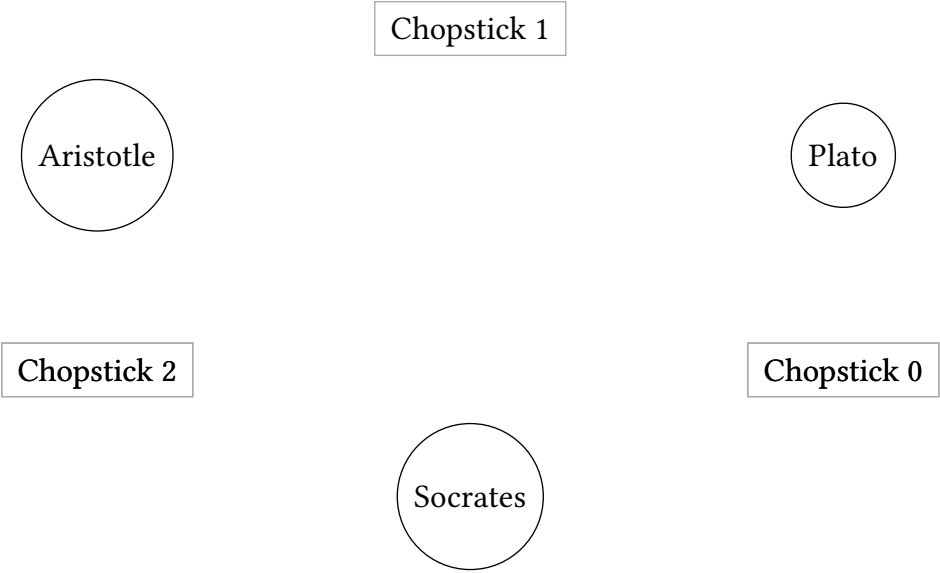


8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

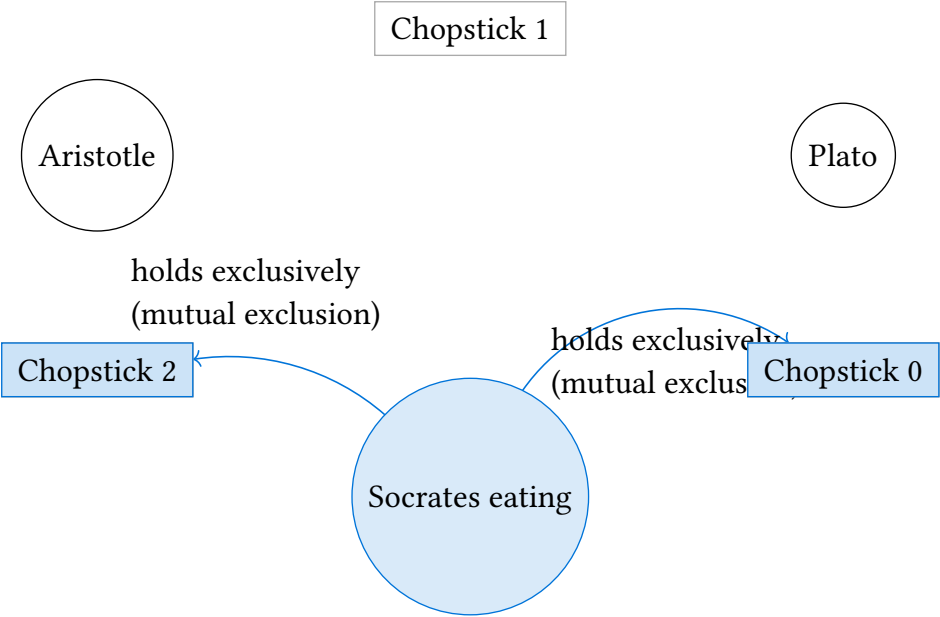


8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

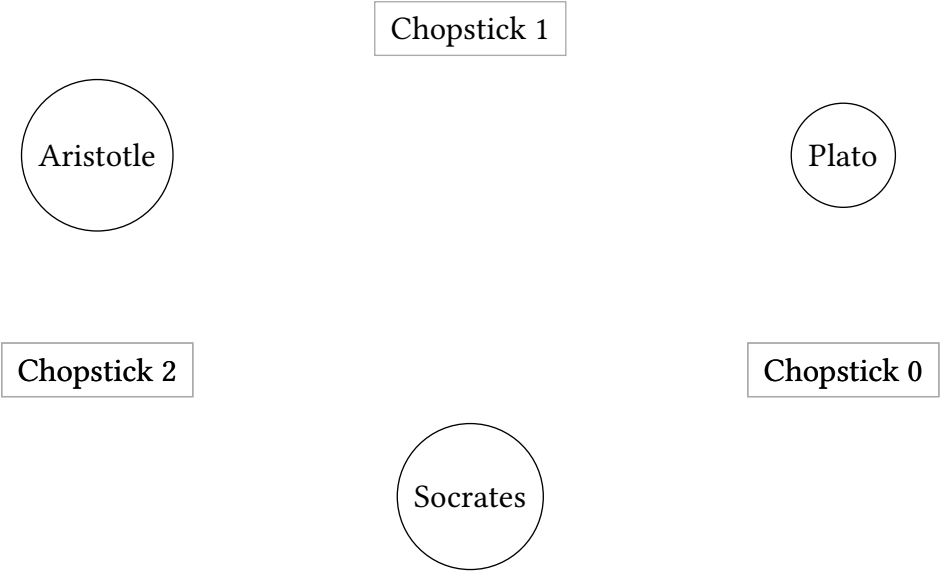


8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

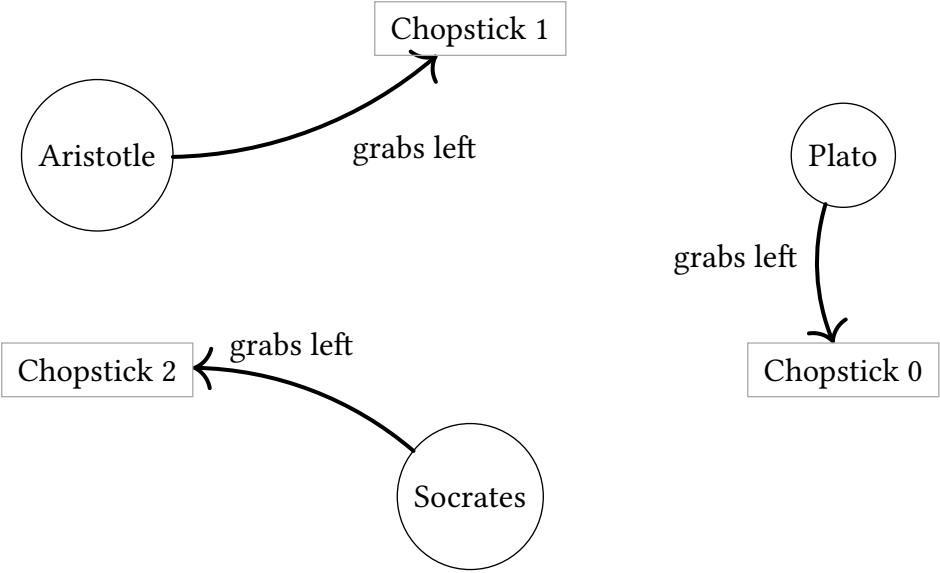


8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

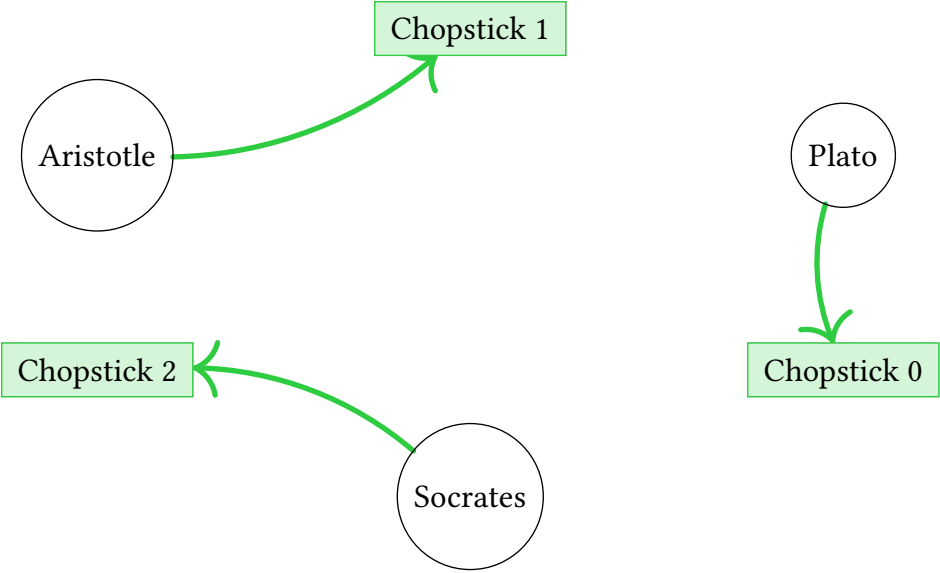


8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

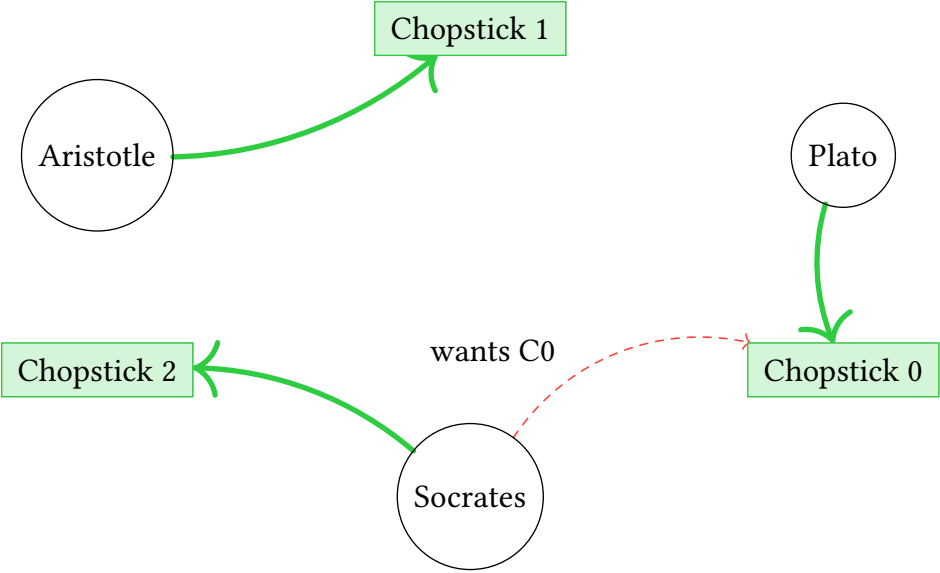


8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

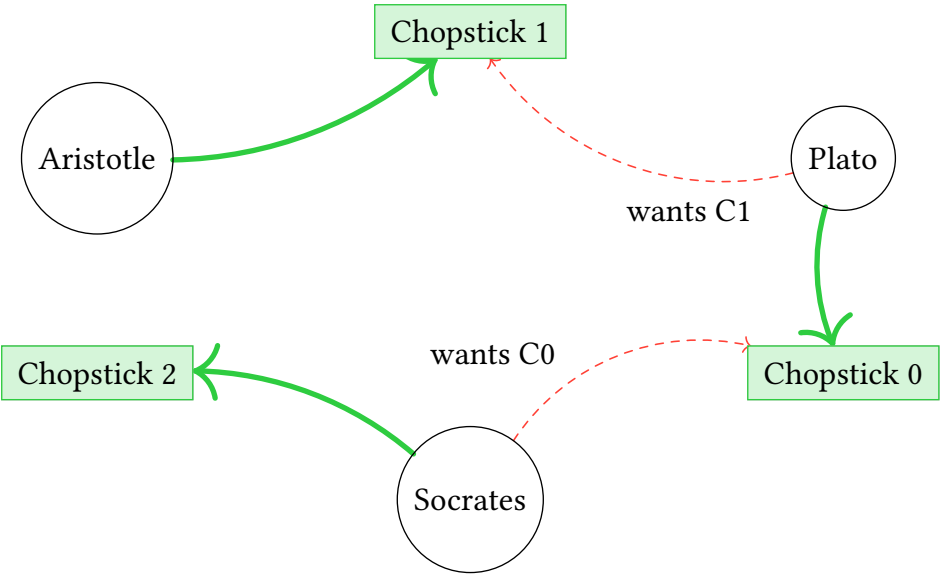


8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

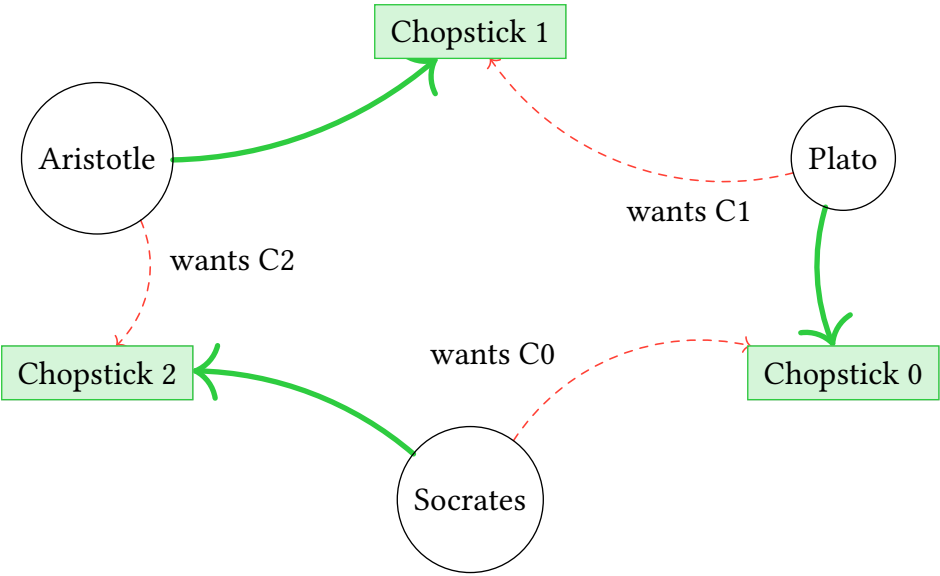


8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

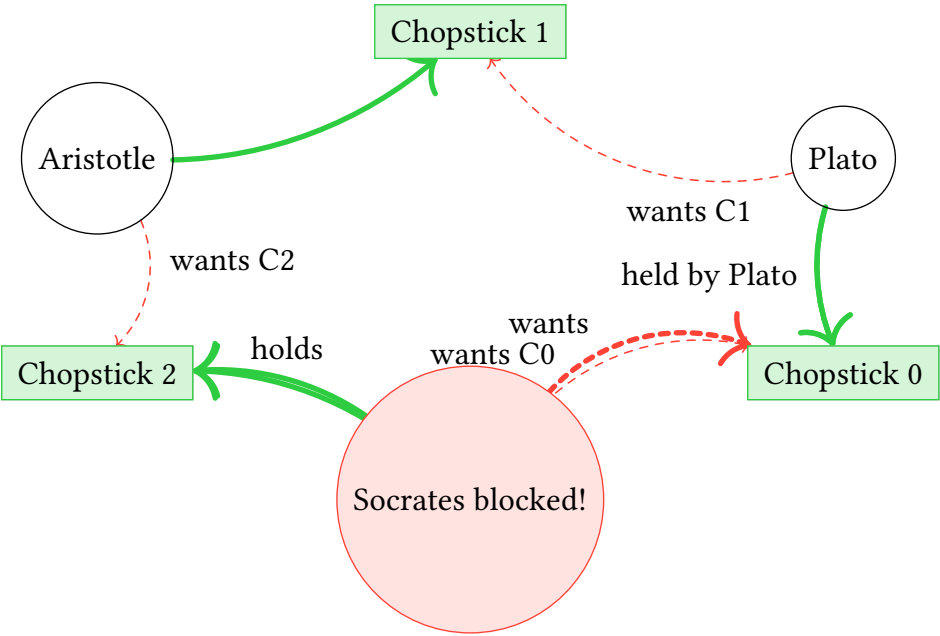


8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

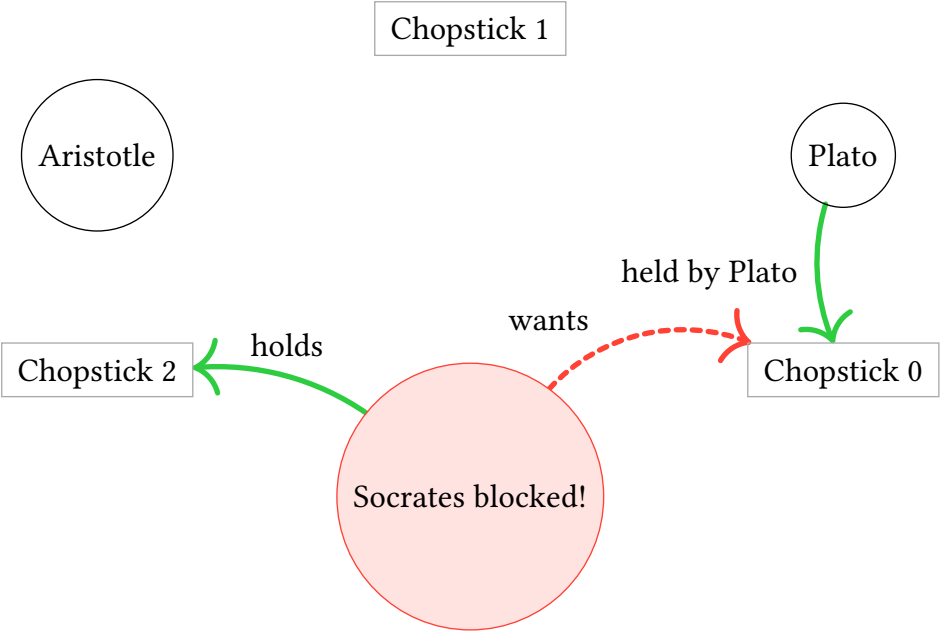


8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly



Legend:

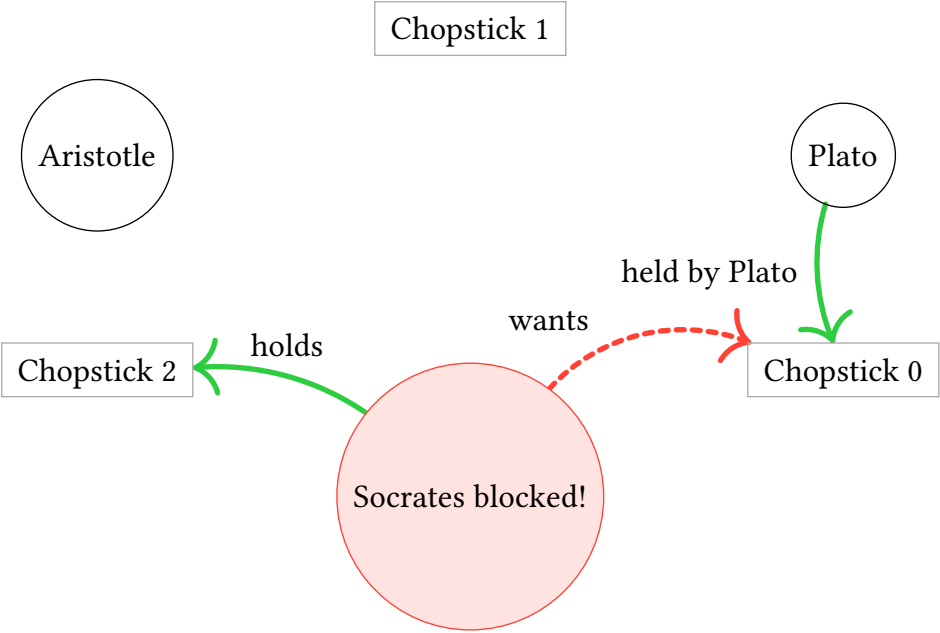
- Trying to grab
- Holding successfully
- - - Wants but blocked

8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly



Legend:

- Trying to grab
- Holding successfully
- - - Wants but blocked

Socrates' perspective:

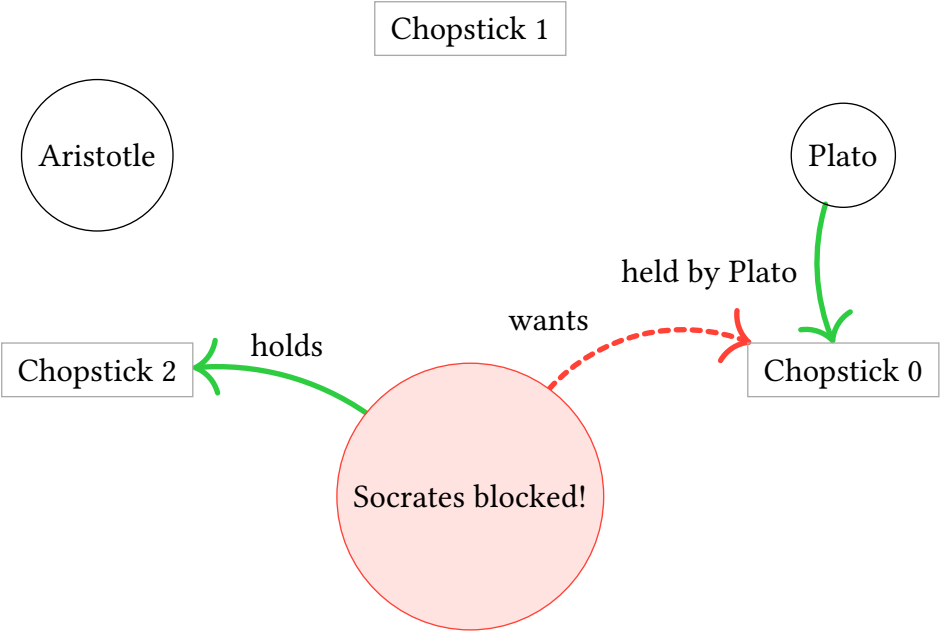
- Holds C2 (green)
- Wants C0 (red dashed)
- But C0 is held by Plato!

8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly



Legend:

- Trying to grab
- Holding successfully
- - - Wants but blocked

Socrates' perspective:

- Holds C2 (green)
- Wants C0 (red dashed)
- But C0 is held by Plato!

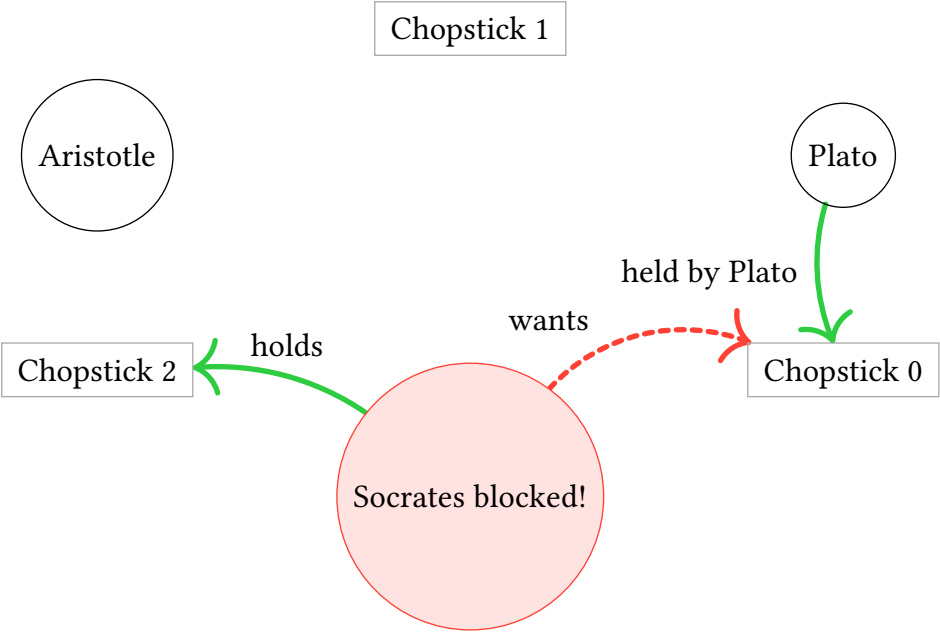
Same for Plato and Aristotle → circular wait → **Deadlock!**

8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly



Legend:

- Trying to grab
- Holding successfully
- - - Wants but blocked

Socrates' perspective:

- Holds C2 (green)
- Wants C0 (red dashed)
- But C0 is held by Plato!

Same for Plato and Aristotle → circular wait → **Deadlock!**

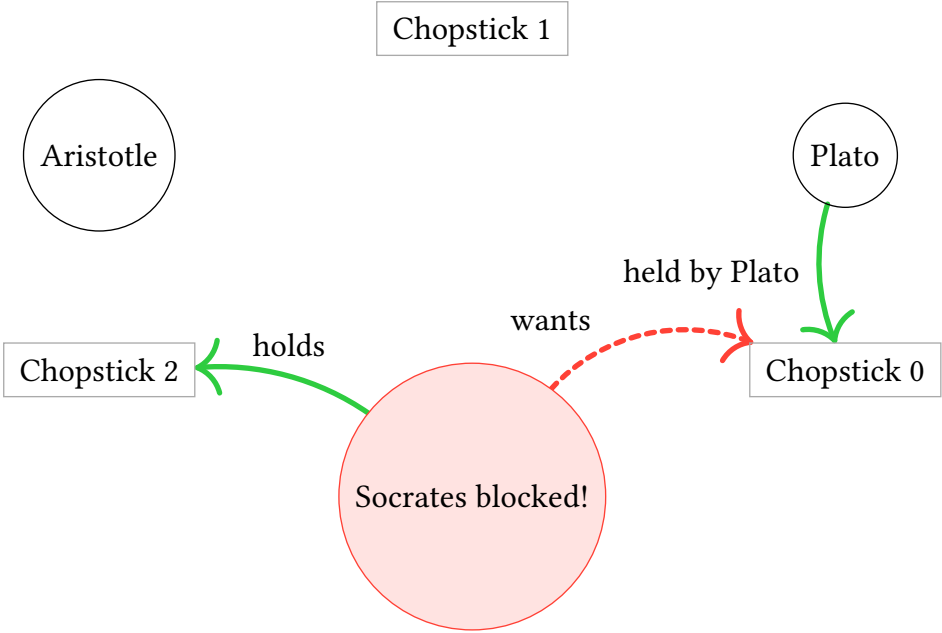
Which Coffman condition should we break?

8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly



Legend:

- Trying to grab
- Holding successfully
- - - Wants but blocked

Socrates' perspective:

- Holds C2 (green)
- Wants C0 (red dashed)
- But C0 is held by Plato!

Same for Plato and Aristotle → circular wait → **Deadlock!**

Which Coffman condition should we break?

Circular wait: change lock order for one philosopher.

8.2. Assignment

Solve the “Dining Philosophers” problem with async tasks instead of threads.

- Revisit the standard thread-based version in `session6/examples/philosophers.rs`
- Complete exercise code in `session7/examples/philosophers.rs`.

Solutions are provided as `*-solution.rs` files.

8.3. Additional bonus

8. Exercise: async philosophers

Do we still need the asymmetry (the last philosopher switching hands)?

8.3. Additional bonus

Do we still need the asymmetry (the last philosopher switching hands)?

Yes, because all philosophers are still working in parallel in separate tasks, so we may have a deadlock.

How to solve this problem with only a single async task? Do we still have deadlocks

Do we still need the asymmetry (the last philosopher switching hands)?

Yes, because all philosophers are still working in parallel in separate tasks, so we may have a deadlock.

How to solve this problem with only a single async task? Do we still have deadlocks

No, since philosophers access sequentially.